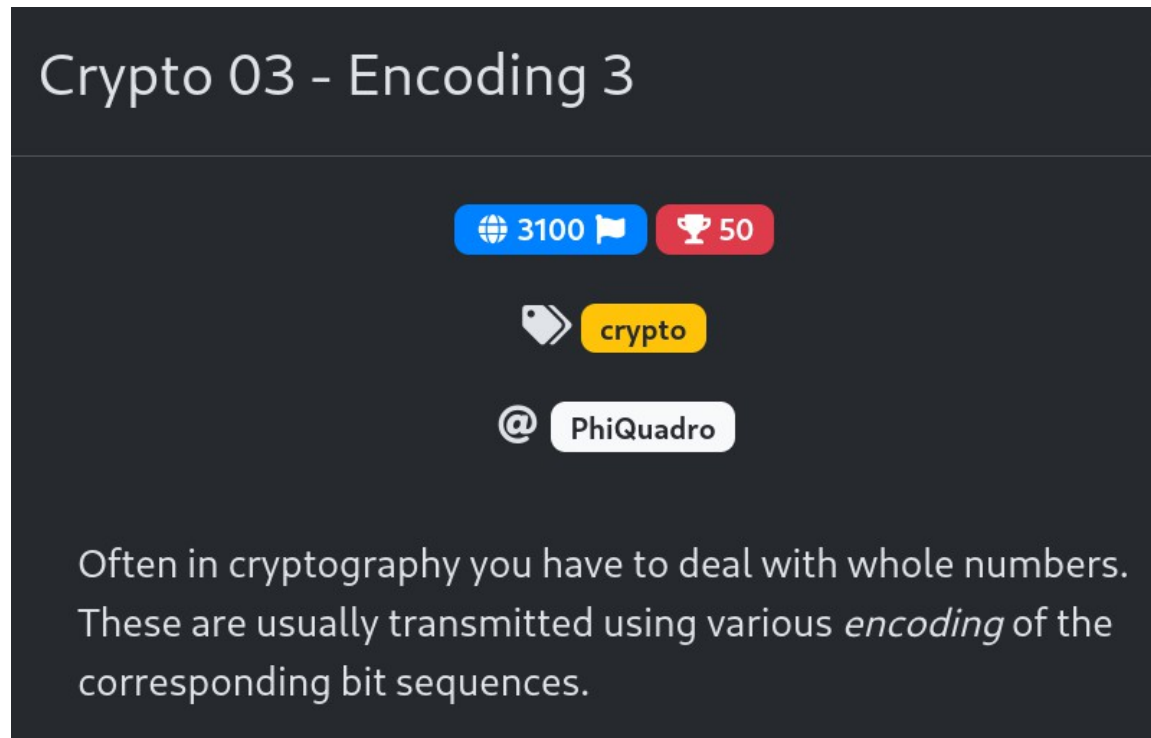


OLlcyber Introduction

Crypto 03 – Encoding 3



In this challenge, we're asked to decode two objects into bytes: a base64-encoded string, as well as a large integer

Crypto 03 – Encoding 3

What are Base64 Strings?

Base 64 Encoding Character Set

abcdefghijklmnopqrstuvwxyz
ABCDEFGHIJKLMNOPQRSTUVWXYZ
0123456789+ /

Base64 is a common encoding system which transforms bytes into alphanumeric strings, which may include the + and / characters

Crypto 03 – Encoding 3

What are Base64 Strings?

Sample Base 64 Encoding Strings

test → dGVzdA==

HackerFrogs → SGFja2VyRnJvZ3M=

123 → MTIz

Here are some sample base64-encoded strings,
as well as their original ASCII forms

Crypto 03 – Encoding 3

What are Base64 Strings?

Sample Base 64 Encoding Strings

test → dGVzdA==

HackerFrogs → SGFja2VyRnJvZ3M=

123 → MTIz

Note that base64-encoded strings must always be a length that is divisible by 4, and if not, a number of = signs will be added to the end of the string until the length is divisible by 4

Crypto 03 – Encoding 3

What is Integer to Byte Conversion?

664813035583918006462745898431981286737635929725

This challenge also requires us to convert a large integer to bytes

Crypto 03 – Encoding 3

What is Integer to Byte Conversion?

ASCII to Decimal Integer Conversion

ASCII	→	ABC
Hex String	→	414243
Decimal	→	4276803

Any byte string can be converted to a decimal number, but first the byte string must be converted to a hexadecimal string, then that hex string is converted to a decimal integer

Crypto 03 – Encoding 3

CyberChef Solve (Part 1)

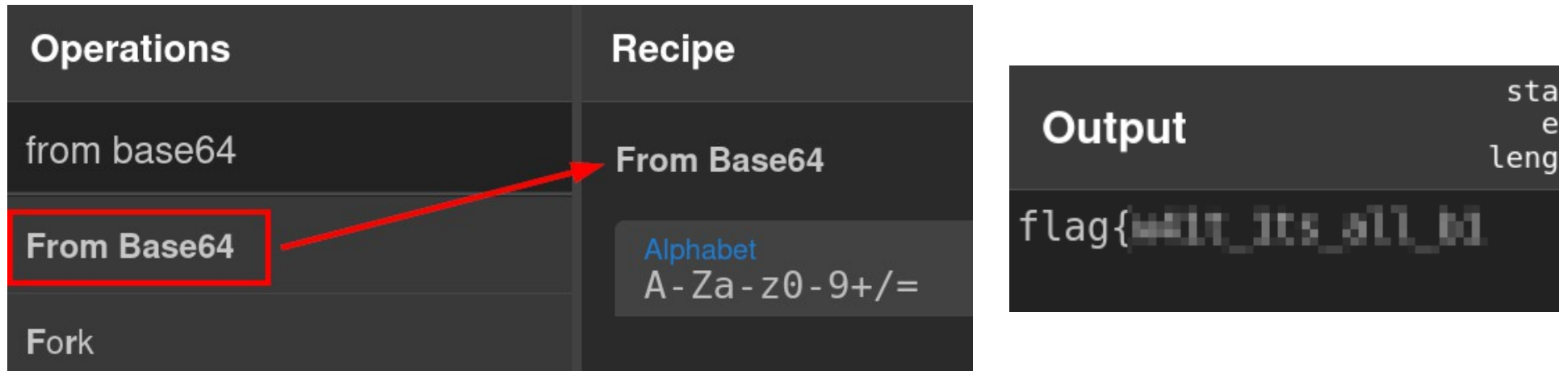
Input

664813035583918006462

To solve this challenge in CyberChef, first we need to copy the base64 string, then paste it into the `Input` window

Crypto 03 – Encoding 3

CyberChef Solve (Part 1)



The screenshot shows the CyberChef interface with three main panels: Operations, Recipe, and Output.

- Operations:** A list of available operations. 'From Base64' is highlighted with a red box, and a red arrow points from it to the Recipe panel.
- Recipe:** Shows the current recipe steps. 'From Base64' is added as the first step. Below it, the 'Alphabet' is set to 'A-Za-z0-9+/'.
- Output:** Displays the result of the operation. The text 'flag{wait_it's_all_b1' is visible, with the last part being partially obscured by a blurred area.

Then, in the `Operations` window, search for the `To Base64` operation, then drag that selection into the `Recipe` window, the output will be in the `Output` window

Crypto 03 – Encoding 3

CyberChef Solve (Part 2)

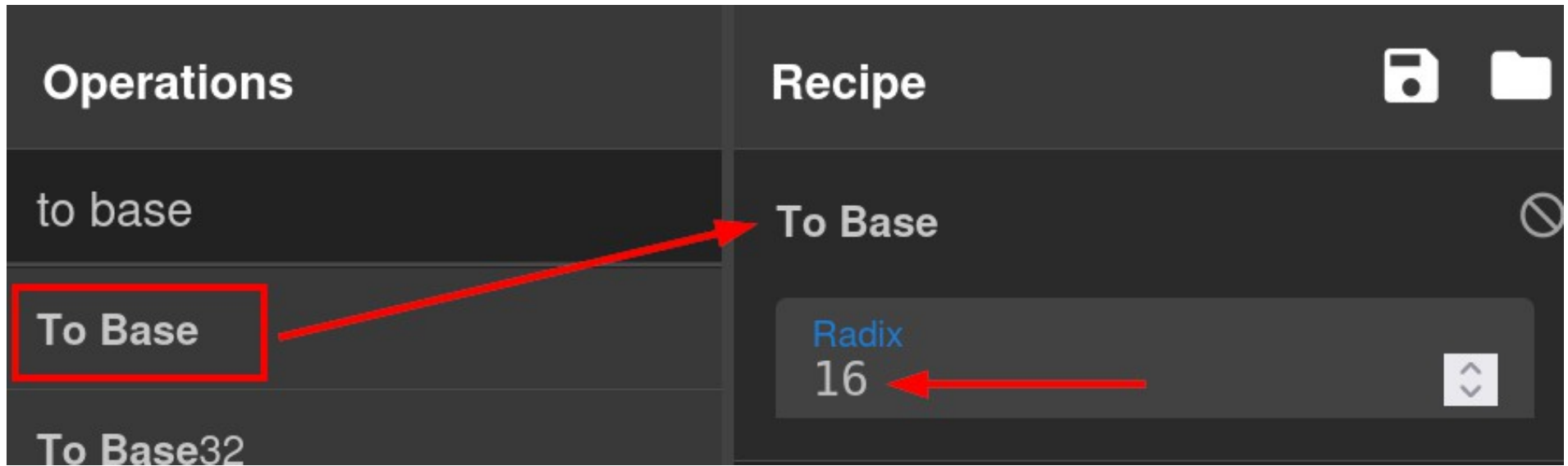
Input

664813035583918006462

For part 2, first we need to copy the large integer,
then paste it into the `Input` window

Crypto 03 – Encoding 3

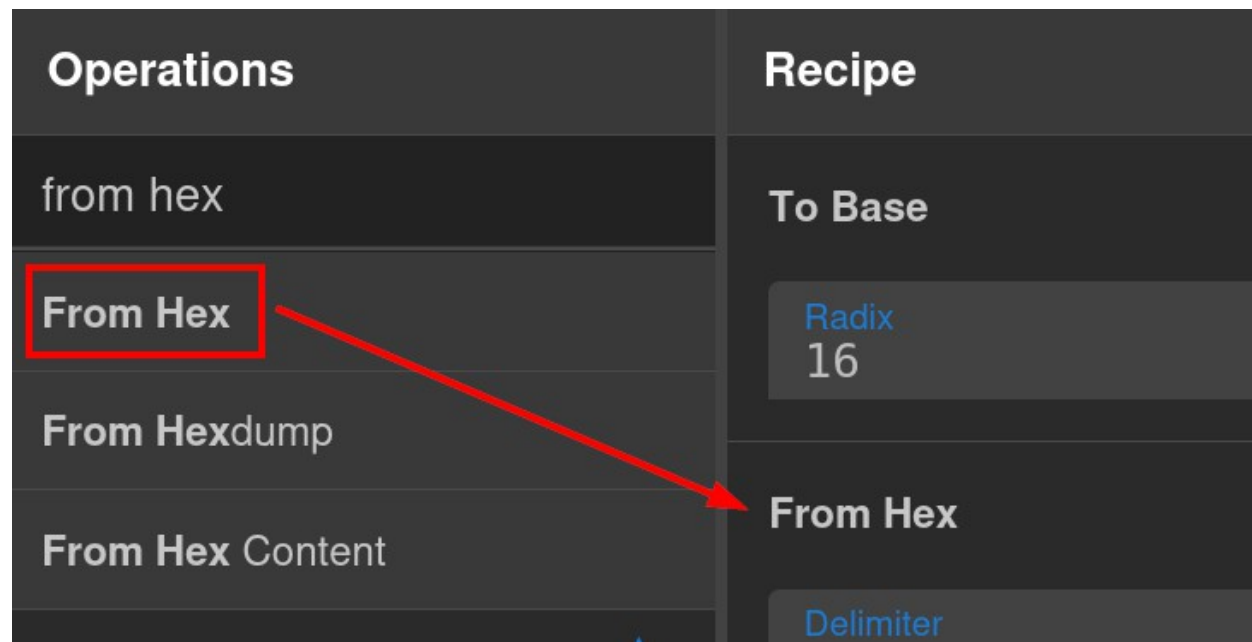
CyberChef Solve (Part 2)



Then, in the `Operations` window, search for the `To Base` operation, then drag that selection into the `Recipe` window, and adjust `Radix` to 16

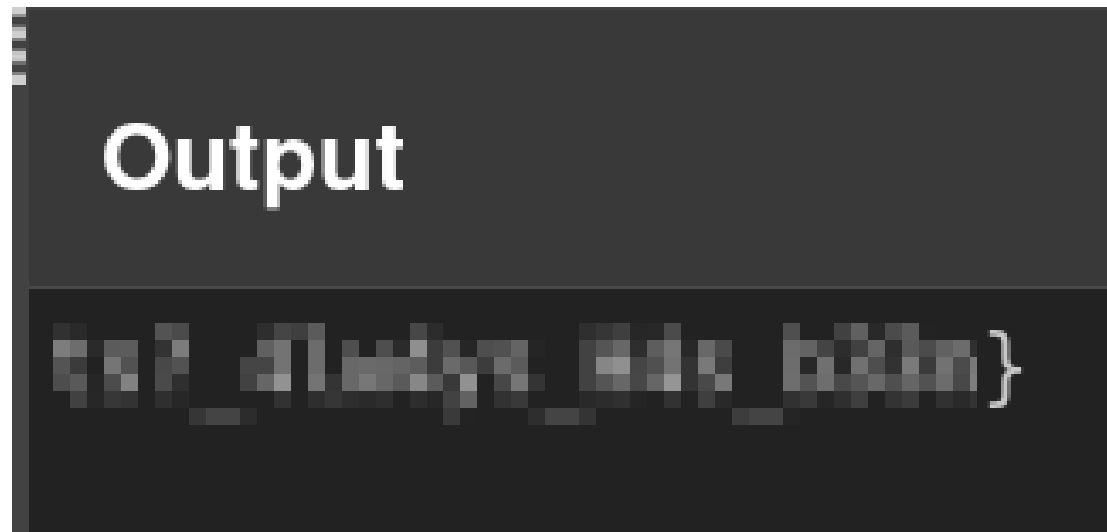
Crypto 03 – Encoding 3

CyberChef Solve (Part 2)



Then, in the `Operations` window, search for the `From Hex` operation, then drag that selection into the `Recipe` window below the `To Base` operation

Crypto 03 – Encoding 3 CyberChef Solve (Part 2)



The operations will convert the integer to hexadecimal, then convert that hex string to bytes, which we can see in the `Output` window

Crypto 03 – Encoding 3

Linux Solve (Part 1)

```
echo -n 'Zmxh...' | base64 -d
```

We can solve this challenge using Linux commands by using the `echo` and `base64` commands

Crypto 03 – Encoding 3

Linux Solve (Part 1)

```
echo 'Zmxh...' | base64 -d
```

First we use the `echo` command to provide input for the next command

Crypto 03 – Encoding 3

Linux Solve (Part 1)

```
echo 'Zmxh...' | base64 -d
```

Then we use the `base64` command with the `-d` argument to decode from base64 to bytes

Crypto 03 – Encoding 3

Linux Solve (Part 2)

```
echo "obase=16; 6648..." | bc | xxd  
-r -p
```

For the second part, we use a combination of the
echo, bc, and xxd commands

Crypto 03 – Encoding 3

Linux Solve (Part 2)

```
echo "obase=16; 6648..." | bc | xxd  
-r -p
```

First we use `echo`, which we use to pass the output base to the next command, as well as the decimal number itself

Crypto 03 – Encoding 3

Linux Solve (Part 2)

```
echo "obase=16; 6648..." | bc | xxd  
-r -p
```

Next, the `bc` command converts the decimal number to hexadecimal

Crypto 03 – Encoding 3

Linux Solve (Part 2)

```
echo "obase=16; 6648..." | bc | xxd  
-r -p
```

Lastly, we use `xxd` to convert from hexadecimal to bytes. The `-r` and `-p` arguments reverse bytes to hex and present it in printable form, respectively

Crypto 03 – Encoding 3

Python Solve

```
from base64 import b64decode  
  
b64_str = 'ZmxhZ3t3NDF0XzF0c19hbGxfYjE='  
big_int = 664813035583918006462745898431981286737635929725
```

To solve this challenge with Python, we can use the base64 string as a string variable and the large decimal number as an integer variable

Crypto 03 – Encoding 3

Python Solve

```
b64_decode = b64decode(b64_str)
```

We set a decoded base64 string variable using the `b64decode` function

Crypto 03 – Encoding 3

Python Solve

```
req_bytes = (big_int.bit_length() + 7) // 8
```

For the decimal to bytes conversion, we need to calculate the byte length of the big number, by using the `bit_length` function, divided by 8 (8-bits in a byte)

Crypto 03 – Encoding 3

Python Solve

```
int_bytes =  
    big_int.to_bytes  
(req_bytes, byteorder='big')
```

Then we use the integer `to_bytes` method to convert the integer to a bytes object, specifying the `req_bytes` value for the number of bytes, and the `byteorder` parameter set to `'big'` for big-endian byte order

Crypto 03 – Encoding 3

Python Solve

```
print(b64_decode + int_bytes)
```

Finally, we print both parts of the flag